TaoFi Security Audit

：TaoFi

August 1, 2025

Revision 1.0

ChainLight@Theori

Theori, Inc. (“We”) is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

© 2025 ChainLight, Theori. All rights reserved

Table of Contents

TaoFi Security Audit 1
Table of Contents 2
Executive Summary 3
Audit Overview 4
- Scope 4
- Code Revision 4
- Severity Categories 5
- Status Categories 6
- Finding Breakdown by Severity 7

Findings 8
- Summary 8
- #1 TAOFI-001 Arbitrary call in _executeExternalCall enables contract fund theft 9
- #2 TAOFI-002 Lack of validation on approvalAddress leads to token drain vulnerability 10
- #3 TAOFI-003 Incorrect remote interchain account address calculation in _bridgeAndCall() 11
- #4 TAOFI-004 Insufficient gas payment for cross-chain transfers may leads to fail 13
- #5 TAOFI-005 Mismatched array lengths in remoteCallWithExternalCalls() may leads to transaction failure 15
- #6 TAOFI-006 Incorrect calculation of unstaked TAO amount in unStakeSwapAndBridging() 16
- #7 TAOFI-007 Incorrect stake ownership assumption in TaoSwapAndBridgeToMain.unStakeSwapAndBridging() 18
- #8 TAOFI-008 Missing validation for swapParams.tokenIn 20
- #9 TAOFI-009 Minor Suggestions 21
- Revision History 23

© 2025 ChainLight, Theori. All rights reserved

Executive Summary

Beginning on June 9, 2025, ChainLight conducted a 3-day security audit of the Study Smart Contracts. The audit focused on evaluating the security of the bridging, staking, and interchain account functionalities across the `SwapBridgeAndCallFromMain`, `TaoSwapAndBridgeToMain`, and `SwapAndStake` contracts. The primary goal was to identify vulnerabilities that could lead to unauthorized asset access, incorrect staking behavior, or unsafe interchain calls.

Summary of Findings

The audit revealed a total of **9** issues, categorized by severity as follows:

- **High:** 2 issues
- **Medium:** 4 issues
- **Low:** 2 issues
- **Informational:** 1 issue

© 2025 ChainLight, Theori. All rights reserved

Audit Overview

Scope

Name	TaoFi Security Audit
Target / Version	• Git Repository (`https://github.com/TaoFi-0x/taofi-private`): `36481231e6b5fd667fc92cd071b3aa1a9ec21a9b`
Application Type	Smart contracts
Lang. / Platforms	Smart contracts [Solidity]

Code Revision

N/A

© 2025 ChainLight, Theori. All rights reserved

Severity Categories

Severity	Description
Critical	The attack cost is low (not requiring much time or effort to succeed in the actual attack), and the vulnerability causes a high-impact issue. (e.g., Effect on service availability, Attacker taking financial gain)
High	An attacker can succeed in an attack which clearly causes problems in the service's operation. Even when the attack cost is high, the severity of the issue is considered "high" if the impact of the attack is remarkably high.
Medium	An attacker may perform an unintended action in the service, and the action may impact service operation. However, there are some restrictions for the actual attack to succeed.
Low	An attacker can perform an unintended action in the service, but the action does not cause significant impact or the success rate of the attack is remarkably low.
Informational	Any informational findings that do not directly impact the user or the protocol.
Note	Neutral information about the target that is not directly related to the project's safety and security.

© 2025 ChainLight, Theori. All rights reserved

Status Categories

Status	Description
Reported	ChainLight reported the issue to the client.
WIP	The client is working on the patch.
Patched	The client fully resolved the issue by patching the root cause.
Mitigated	The client resolved the issue by reducing the risk to an acceptable level by introducing mitigations.
Acknowledged	The client acknowledged the potential risk, but they will resolve it later.
Won't Fix	The client acknowledged the potential risk, but they decided to accept the risk.

© 2025 ChainLight, Theori. All rights reserved

Finding Breakdown by Severity

Category	Count	Findings
Critical	0	• N/A
High	2	• `TAOFI-001` • `TAOFI-002`
Medium	4	• `TAOFI-003` • `TAOFI-004` • `TAOFI-006` • `TAOFI-007`
Low	2	• `TAOFI-005` • `TAOFI-008`
Informational	1	• `TAOFI-009`
Note	0	• N/A

© 2025 ChainLight, Theori. All rights reserved

Findings

Summary

#	ID	Title	Severity	Status
1	TAOFI-001	Arbitrary call in `_executeExternalCall` enables contract fund theft	High	Patched
2	TAOFI-002	Lack of validation on `approvalAddress` leads to token drain vulnerability	High	Patched
3	TAOFI-003	Incorrect remote interchain account address calculation in `_bridgeAndCall()`	Medium	Patched
4	TAOFI-004	Insufficient gas payment for cross-chain transfers may leads to fail	Medium	Patched
5	TAOFI-005	Mismatched array lengths in `remoteCallWithExternalCalls()` may leads to transaction failure	Low	Patched
6	TAOFI-006	Incorrect calculation of unstaked TAO amount in `unStakeSwapAndBridging()`	Medium	Patched
7	TAOFI-007	Incorrect stake ownership assumption in `TaoSwapAndBridgeToMain.unStakeSwapAndBridging()`	Medium	Patched
8	TAOFI-008	Missing validation for `swapParams.tokenIn`	Low	Patched
9	TAOFI-009	Minor Suggestions	Informational	Patched

© 2025 ChainLight, Theori. All rights reserved

#1 `TAOFI-001` Arbitrary call in `_executeExternalCall` enables contract fund theft

ID	Summary	Severity
`TAOFI-001`	Allowing arbitrary execution in `_executeExternalCall` may lead to the theft of contract funds.	**High**

Description

The `lifiSwapBridgeAndCall()` function invokes `_executeExternalCall()`, which allows arbitrary calls to any target with arbitrary calldata. This can be exploited to execute malicious logic, potentially resulting in the theft of fee tokens or other contract-held assets.

Impact

High

An attacker may perform unauthorized external calls to drain funds or manipulate contract state, leading to significant financial loss.

Recommendation

Remove `_executeExternalCall()` and integrate the swap execution directly with `LiFiRouter` to eliminate arbitrary call vectors.

Remediation

Patched

It has been patched as recommended.

© 2025 ChainLight, Theori. All rights reserved

#2 `TAOFI-002` Lack of validation on `approvalAddress` leads to token drain vulnerability

ID	Summary	Severity
`TAOFI-002`	Unrestricted `approvalAddress` in `SwapBridgeAndCallFromMain.lifiSwapBridgeAndCall()` allows token theft.	**High**

Description

In the `SwapBridgeAndCallFromMain.lifiSwapBridgeAndCall()` function, an attacker can specify their own address instead of the `LiFiRouter` as the `_approvalAddress`. This allows them to exploit `IERC20(_fromToken).safeApprove(_approvalAddress, _fromAmount)` to approve tokens to themselves and steal tokens held by the contract.

Impact

High

By setting `_approvalAddress` to an attacker-controlled address, the contract may unknowingly approve tokens, which the attacker can then transfer and steal.

Recommendation

Restrict `_approvalAddress` to only allow the trusted `LiFiRouter` address.

Remediation

Patched

The issue has been resolved since the `SwapBridgeAndCallFromMain` contract no longer collects fees.

© 2025 ChainLight, Theori. All rights reserved

#3 `TAOFI-003` Incorrect remote interchain account address calculation in `_bridgeAndCall()`

ID	Summary	Severity
`TAOFI-003`	The `SwapBridgeAndCallFromMain._bridgeAndCall()` function may calculate an incorrect remote interchain account (ICA) address because it does not use the provided router and ISM overrides.	**Medium**

Description

In `_bridgeAndCall()`, the `ica.getRemoteInterchainAccount()` function is called to determine the remote ICA address. However, it only uses `DESTINATION_CHAIN_ID`, `address(this)`, and `userSpecificSalt` as parameters.

```
    function _bridgeAndCall(RemoteCallsParams calldata _params, uint256 valu
eSpent) internal {
        ...
        IInterchainAccountRouter ica = IInterchainAccountRouter(interchainAc
countRouter);
        bytes32 userSpecificSalt = bytes32(uint256(uint160(msg.sender)));

        {
            address userIcaOnDestination =
                ica.getRemoteInterchainAccount(DESTINATION_CHAIN_ID, address
(this), userSpecificSalt);

            // Bridge
            IBridge(_bridge).transferRemote{value: 1 wei}(
                DESTINATION_CHAIN_ID, bytes32(uint256(uint160(userIcaOnDesti
nation))), swapAmount
            );
        }
```

© 2025 ChainLight, Theori. All rights reserved

```
        IInterchainAccountRouterWithOverrides(interchainAccountRouter).callR
emoteWithOverrides{
            value: msg.value - valueSpent - 1 wei
        }(
            DESTINATION_CHAIN_ID,
            _params.router, // Or your bytes32 router override
            _params.ism, // Or your bytes32 ISM override
            _params.calls,
            _params.hookMetadata,
            userSpecificSalt
        );
    }
```

However, unlike `getRemoteInterchainAccount()`, the call to `interchainAccountRouter.callRemoteWithOverrides()` uses `params.router` and `params.ism`. If the default router and default ISM registered in the `interchainAccountRouter` differ from the `router` and `ism` provided in `params`, different remote account addresses may be computed. In this case, the address that actually receives the funds and the address that receives the commands will be different.

Impact

Medium

The remote ICA address that receives funds and the remote ICA address that receives commands may differ, which can result in the TAO transfer not being properly staked.

Recommendation

It is recommended to modify the `getRemoteInterchainAccount()` function to `ica.getRemoteInterchainAccount(address(this), _params.router, _params.ism, userSpecificSalt)` to ensure that the address receiving the funds and the address executing the command are the same.

Remediation

Patched

The issue has been resolved as recommended.

© 2025 ChainLight, Theori. All rights reserved

#4 `TAOFI-004` Insufficient gas payment for cross-chain transfers may leads to fail

ID	Summary	Severity
`TAOFI-004`	In the `SwapBridgeAndCallFromMain` and `TaoSwapAndBridgeToMain` contracts, insufficient gas is provided when calling `transferRemote()`, which significantly increases the likelihood of cross-chain transfer failures.	**Medium**

Description

In the `_bridgeAndCall()` function, the `transferRemote()` call initiates a cross-chain transfer. However, it provides a gas payment of `msg.value` which is hardcoded to 1, regardless of the actual cost. Similarly, the `transferRemote()` call in the `TaoSwapAndBridgeToMain` contract is also susceptible to this issue. Cross-chain operations require a specific gas fee to be paid on the source chain to cover the costs of execution on the destination chain. A hardcoded value of 1 is highly likely to be insufficient for this purpose.

Impact

Medium

Insufficient gas payment will cause the cross-chain transaction to fail on the destination chain. The user can resolve this issue by paying additional gas for the message, but this leads to a poor user experience.

Recommendation

The correct gas payment for cross-chain transfers must be dynamically calculated using the `quoteGasPayment()` function. The `IBridge(Bridge).quoteGasPayment(DESTINATION_CHAIN_ID)` function should be used to determine the proper gas payment and ensure that `msg.value` is set to this calculated amount for the `transferRemote()` call. This recommendation also applies to the `IBridge(USDC).transferRemote()` call within the `TaoSwapAndBridgeToMain` contract to ensure reliable cross-chain transfers.

© 2025 ChainLight, Theori. All rights reserved

Remediation

Patched

The issue has been resolved as recommended.

© 2025 ChainLight, Theori. All rights reserved

#5 `TAOFI-005` Mismatched array lengths in `remoteCallWithExternalCalls()` may leads to transaction failure

ID	Summary	Severity
`TAOFI-005`	The `SwapBridgeAndCallFromMain.remoteCallWithExternalCalls()` function lacks a check to ensure that the lengths of the `preRemoteCalls`, `postRemoteCalls`, and `remoteCalls.calls` arrays are equal.	Low

Description

For the function to operate correctly, the number of entries in `preRemoteCalls`, `postRemoteCalls`, and `remoteCalls.calls` must be identical. Currently, there is no validation to enforce this, which may result in unexpected behavior if the array lengths do not match during execution.

Impact

Low

If the array lengths are not equal, the function will likely fail to execute as intended.

Recommendation

Add a validation check at the beginning of the `remoteCallWithExternalCalls()` function to ensure that the lengths of `preRemoteCalls`, `postRemoteCalls`, and `remoteCalls.calls` are all equal. If they are not, the function should revert with an appropriate error message.

Remediation

Patched

The issue has been resolved with the removal of the `remoteCallWithExternalCalls()` function.

© 2025 ChainLight, Theori. All rights reserved

#6 `TAOFI-006` Incorrect calculation of unstaked TAO amount in `unStakeSwapAndBridging()`

ID	Summary	Severity
`TAOFI-006`	The `TaoSwapAndBridgeToMain.unStakeSwapAndBridging()` incorrectly includes the contract's pre-existing TAO balance when calculating the amount of unstaked TAO.	Medium

Description

In the `unStakeSwapAndBridging()` function, `taoAmount` is calculated after calling the `removeStakeLimit()` function. The value is computed using `uint256 taoAmount = address(this).balance - msg.value;`, which can lead to incorrect results if the contract already holds TAO tokens.

Impact

Medium

If the contract has a pre-existing TAO balance, the calculated unstaked TAO amount may be overestimated. This can lead to greater slippage during off-chain swaps from TAO to USDC if the minimum expected USDC amount (`_minAmount`) is calculated without accounting for the contract's existing TAO balance. In other words, although the amount of TAO used as `tokenIn` increases, the `mintAmount` remains the same, resulting in a loss for the user during the swap.

Recommendation

It is recommended to record the contract's TAO balance before calling `removeStakeLimit()` using `uint256 oldBalance = address(this).balance;`. After the call, `taoAmount` should be calculated as `address(this).balance - oldBalance`.

Remediation

Patched

© 2025 ChainLight, Theori. All rights reserved

`TaoSwapAndBridgeToMain` has been replaced with the `StakeManager` contract, and the patch has been applied in `_removeStake()` as recommended.

© 2025 ChainLight, Theori. All rights reserved

#7 `TAOFI-007` Incorrect stake ownership assumption in `TaoSwapAndBridgeToMain.unStakeSwapAndBridging()`

ID	Summary	Severity
`TAOFI-007`	In the `TaoSwapAndBridgeToMain` contract, the call to `IStaking(ISUBTENSOR_STAKING_ADDRESS).removeStakeLimit()` reduces the stake of the contract itself (used as `coldKey`), not `msg.sender`. However, the current logic implies that it is withdrawing the sender's stake.	**Medium**

Description

`TaoSwapAndBridgeToMain.unStakeSwapAndBridging()` calls `IStaking(ISUBTENSOR_STAKING_ADDRESS).removeStakeLimit()` to unstake TAO. However, `removeStakeLimit` decreases the alpha stake of the `TaoSwapAndBridgeToMain` contract address, not that of `msg.sender`. For this function to operate correctly, `msg.sender` must first transfer alpha tokens to the `TaoSwapAndBridgeToMain` contract and then atomically call `unStakeSwapAndBridging()`. Alternatively, the function should be invoked via `delegateCall` to `TaoSwapAndBridgeToMain.unStakeSwapAndBridging()`.

Impact

Medium

The `TaoSwapAndBridgeToMain` contract performs unstaking without receiving alpha tokens directly from the user, which may lead to the function not operating as intended. While it is possible for the function to behave correctly if the user first transfers alpha tokens to the contract and then calls `unStakeSwapAndBridging()`, this process must be executed atomically. Moreover, such an approach is not feasible on subnets that do not support token transfers.

Although calling the function via `delegateCall` may also enable correct execution, this method is not applicable in cases where the caller is an ICA (Interchain Account), as delegateCall is not supported in that context.

Recommendation

One of the following two approaches should be implemented:

© 2025 ChainLight, Theori. All rights reserved

1. It is recommended to retrieve `_alphaAmount` of alpha tokens from `msg.sender` using `transferTokenFrom()`. However, note that the alpha token's transfer functionality is optional and may not be implemented by all subnet creators, making this approach potentially infeasible.
2. It is recommended to enforce that `TaoSwapAndBridgeToMain.unStakeSwapAndBridging()` can only be called via `delegateCall` by adding the following validation: `require(address(this) != SELF, "Function must be called via delegatecall");`

Remediation

Patched

`TaoSwapAndBridgeToMain` has been replaced with the `StakeManager` contract, and the above issue has been resolved by distributing a separate alpha ERC20 token to staking users and retrieving it via `transferFrom` during unstaking. Since the newly created alpha token is different from the subnet's alpha token, it can be transferred at any time.

© 2025 ChainLight, Theori. All rights reserved

#8 `TAOFI-008` Missing validation for `swapParams.tokenIn`

ID	Summary	Severity
`TAOFI-008`	The comment for the `swapAndStake()` suggests that a specific token (USDC) should be used, but the code allows any token, creating a discrepancy between the documentation and the implementation.	Low

Description

A comment in the code states "Take USDC," implying that the `swapParams.tokenIn` parameter should be a USDC token. However, there is no corresponding validation logic to enforce this.

Impact

Low

This is a documentation issue that could lead to user confusion or a misunderstanding of the function's intended behavior.

Recommendation

It is recommended to add a check to enforce that `swapParams.tokenIn == USDC`.

Remediation

Patched

The issue has been resolved as recommended.

© 2025 ChainLight, Theori. All rights reserved

#9 `TAOFI-009` Minor Suggestions

ID	Summary	Severity
`TAOFI-009`	The description includes multiple suggestions for preventing incorrect settings caused by operational mistakes, mitigating potential issues, and improving code maturity and readability.	Informational

Description

1. In `SwapBridgeAndCallFromMain`, the `setBridgeToken()`, `setBridge()`, and `setInterchainAccountRouter()` functions lack validation to ensure that input addresses are for active contracts.

2. The `SwapBridgeAndCallFromMain.interchainAccountRouter.callRemoteWithOverrides()` function may fail on the destination chain if the gasLimit is too low. Additionally, if the refundAddress is not properly configured, unused funds may not be returned.

3. The `TaoSwapAndBridgeToMain.unStakeSwapAndBridging()` function contains an unnecessary call to `IERC20(USDC).safeApprove(USDC, 0);`. This line of code can be removed to improve efficiency and code clarity.

4. A comment in `TaoSwapAndBridgeToMain.unStakeSwapAndBridging()` implies that the function only supports the Ethereum chain. However, the presence of `_destinationChainId` allows other chains, creating a misleading comment.

5. The `SwapAndStake.swapAndStake()` function is marked as payable but does not require this modifier, as it does not handle incoming Ether.

6. The `SwapAndStake .swapAndStake()` function does not currently validate the `swapParams.tokenOut` to ensure it is a supported token type (i.e., native TAO or WTAO).

Impact

Informational

Recommendation

Consider applying the suggestions in the description above.

© 2025 ChainLight, Theori. All rights reserved

Remediation

Patched

It has been patched as recommended, except for Issue 2.

© 2025 ChainLight, Theori. All rights reserved

Revision History

Version	Date	Description
1.0	August 1, 2025	Initial version

© 2025 ChainLight, Theori. All rights reserved

Theori, Inc. (“We”) is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

ChainLight

© 2025 ChainLight, Theori. All rights reserved